

第三次挑战：比较并改进 RayServe 路由

摘要

本实验在同一台 4×RTX 4090 服务器上，用 `ray.cluster_utils.Cluster` 建立 **1 个 Ray head 节点 + 4 个逻辑 worker 节点**，每个 worker 节点运行 1 个 SGLang 服务（各自占用一张 GPU、各自加载一份 Qwen3-4B-Instruct-2507 权重），并在 Ray Serve 上部署 **4 个 Replica**，使每个 Replica 固定落在一个 worker 节点、固定转发到该节点的 SGLang 后端。客户端只访问同一个 Ray Serve HTTP 入口。

在模型、SGLang 参数、四副本拓扑和压测端配置完全一致的前提下，通过同一个入口回放课程固定负载（64 条预热 + 2048 条测量请求，SHA-256 校验通过），比较了 A / B1 / B2 / C / D 五组路由配置（另有 2 个补充标定点）。每组实验前都重新部署 Ray Serve 配置、对四个 SGLang 后端逐个调用 `POST /flush_cache`，因此每组都从冷缓存开始。

主要结论

1. **A 组默认值把吞吐压到 12.1 req/s**：`max_ongoing_requests=5` 把每个 Replica 的在途请求数限制在 5，全服务的请求级并发与调度容量只有 $4 \times 5 = 20$ ，大量请求堵在 Ray Serve 代理侧排队，TTFT p95 高达 132.5 s。
2. **B 组选定 `max_ongoing_requests=64`**：吞吐 40.2 req/s（A 的 3.3 倍），TTFT p95 降到 18.9 s（比 A 低 85.7%）。再抬到 128 不再有收益（39.9 req/s、p95 18.9 s），64 已在吞吐平台期。
3. **C 组（严格一致性哈希）用缓存换来了热点**：缓存命中率 95.7%、实际 Prefill token 只有 13.8 万（A 的 23.6%），但超热前缀族的 768 条请求全部压在一个后端，最大后端占比 58.0%，该后端排队把吞吐拖到 19.8 req/s、TTFT p95 恶化到 70.7 s，GPU 平均利用率仅 44.6%。
4. **D 组（自定义前缀亲和 + 负载保护）取得更好的平衡**：缓存命中率 85.0%、实际 Prefill token 48.5 万（比 B2 少 17.8%），吞吐 40.0 req/s 与 B2 持平，TTFT p95 22.3 s（比 B2 高 3.4 s），四个后端分布仍在 24.2%–25.8% 之间，没有热点。
4. 五组的 `dispatch_lag_s` p95 ≤ 2.5 ms、`client_queue_s` p95 ≤ 0.4 ms，压测端没有成为瓶颈；所有数字都能对应到 `results/` 下的原始 CSV/JSON。

1. 实验环境与拓扑

1.1 服务结构

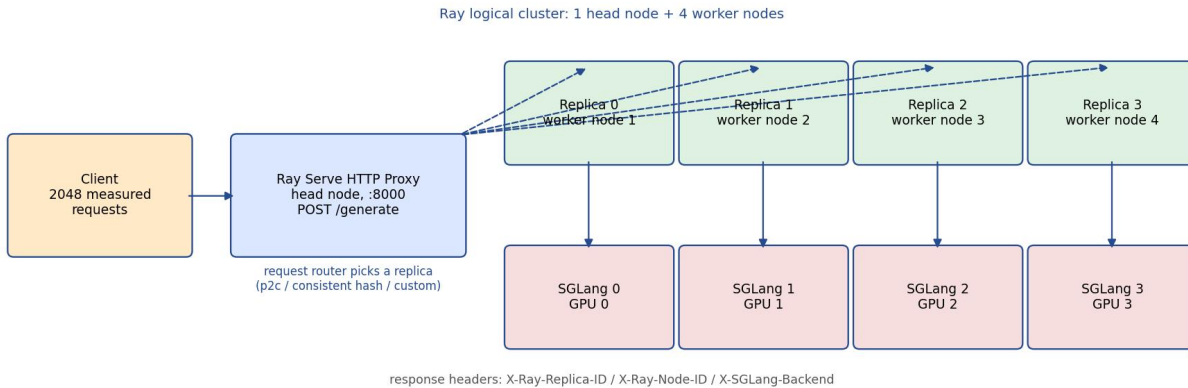


图 1 四副本服务拓扑：客户端 → Ray Serve HTTP 代理 → 4 个 Replica（4 个 worker 节点）→ 4 个 SGLang 后端（4 张 GPU）

Ray Serve 负责选择 Replica，SGLang 负责组批、推理与 KV cache。四个 Replica 与四个 SGLang 后端是一一绑定的：Replica 启动时读取 `run/topology.json`，用自己所在的 Ray 节点 ID 查出该节点的 GPU 编号与 SGLang 地址，因此「副本 $i \leftrightarrow$ worker 节点 $i \leftrightarrow$ GPU $i \leftrightarrow$ SGLang i 」是确定性映射，而不是碰运气。

对应要求的三个响应头由 Replica 在转发前写入：X-Ray-Replica-ID（Ray Serve 的 replica actor id）、X-Ray-Node-ID（Ray 节点 ID）、X-SGLang-Backend（sglang-0..3）。流量发生器会校验这三个头存在，并统计涉及的 Replica / 节点 / 后端个数。

1.2 集群与部署

- 用 `ray.cluster_utils.Cluster(initialize_head=True)` 起 head 节点，再逐个 `add_node()` 起 4 个 worker 节点；每个 worker 节点在启动时带私有自定义资源 `worker_i = 1.0`，用于把 SGLang 进程与放置组 bundle 精确钉在指定节点上。
- Ray Serve 侧用 `gang_scheduling_config=GangSchedulingConfig(gang_size=4, gang_placement_strategy=GangPlacementStrategy.SPREAD)` 让 4 个 Replica 分散到不同节点；部署后脚本主动探测 `/health`，验证「4 副本 $\leftrightarrow$ 4 节点 $\leftrightarrow$ 4 后端」，不满足就重新部署（最多 6 次）。
- 软件版本：Ray 2.59.0、SGLang 0.5.6.post2、PyTorch 2.9.1+cu128、flashinfer 0.5.3、Python 3.12、驱动 595.71.05。
- SGLang 统一启动参数（五组完全一致，取自四个后端的启动日志第一行）：

```
python -m sglang.launch_server --model-path /root/autodl-tmp/models/Qwen3-4B-Instruct-2507 \
  --host 0.0.0.0 --port 30000 --attention-backend flashinfer --page-size 1 \
  --mem-fraction-static 0.75 --max-running-requests 32 --chunked-prefill-size 4096 \
  --schedule-policy fcfs --enable-cache-report --enable-metrics --trust-remote-code
```

每个实例的 CUDA_VISIBLE_DEVICES 固定为一张卡。该命令行同时写入每轮的 config.json /

summary.json 的 sglang_command 字段（见 1.5 的记录勘误）。

1.5 记录勘误：模型字段与后端参数

- **模型字段：**课程流量发生器 run_workload.py 没有 --model 参数，它总是把自己内置的默认值

（Qwen/Qwen3-0.6B）写进 config.json / summary.json 的顶层 model 字段，因此该字段与

实际服务的模型不一致。四个后端实际加载的是 **Qwen/Qwen3-4B-Instruct-2507**

（依据：sglang-gpu0..3.log 第一行的 --model-path，以及每轮 config.json.metadata.model）。

交付包中该字段已更正为真实模型，原值保留在 model_recorded_by_generator，

并附完整启动命令行；后续脚本 stamp_served_model() 会自动写入真实模型，避免同类错误。

- ``max_running_requests``：正式五组（A/B1/B2/C/D）以及 B3、两个 D 变体都在

``--max-running-requests 32`` 的后端上运行（依据同上启动日志；集群在正式实验前以 32 重启过）。

本文所有数字均来自这一配置。

- 详细勘误与核对方法见交付包中的 results/target3/NOTES.md。

1.3 负载

课程固定负载 mooncake_prefix_workload_v2_seed2026.jsonl（SHA-256 f154ba95...5b03，校验通过）：64 个前缀族、64 条预热、2048 条测量请求，输入合计 3,233,280 token，请求输出合计 268,831 token；测量阶段分 steady(512, 40 req/s) / burst(1024, 120 req/s) / recovery(512, 60 req/s)，其中 1 个超热前缀族占 768 条请求且每条生成 190 token（同时制造缓存热点与持续生成压力）。所有组都通过 run_workload.py --policy serve 回放，未修改请求内容、到达时间或顺序（生成器自身会核对负载 SHA-256）。

1.4 五组配置

组别	Ray Serve 路由配置	max_ongoing_requests	--router-name	结果目录
A	默认路由器 (power of two choices)	5	p2c	results/A_default/
B1	默认路由器	16	p2c	results/B_candidates/candidate-1/
B2 (选定)	默认路由器	64	p2c	results/B_candidates/candidate-2/
C	ConsistentHashRouter (num_virtual_nodes=100、 num_fallback_replicas=0 , 会话键 X-Session-Id = 前缀族 ID)	64	consistent_hash	results/C_affinity/
D	自定义 PrefixAffinityRouter (前缀亲和 + 负载保护)	64	prefix_affinity_guarded	results/D_improved/

补充标定点（不作为正式对照组，但其数据用于选择 B 值、说明 D 的参数敏感性）：

标定	说明	结果目录
B3	默认路由器, max_ongoing_requests=128	results/B_candidates/calibration-128/
D'	自定义路由器, 保护阈值 0.7 (更松)	results/D_variants/relaxed-0.7/

2. 主表：五组对照结果

五组全部满足 successful=2048 / failed=0 / incomplete=0、预热 64/64、validation_errors=[], 且四个 Replica、四个节点、四个后端均被覆盖 (results/*/summary.json、deploy.json) 。吞吐量 = 成功请求数 ÷ 测量阶段总耗时。

指标	A (默认, mor=5)	B1 (mor=16)	B2 (mor=64, 选定)	C (一致性哈希)	D (前缀亲和+保护)
成功率	100% (2048/2048)	100%	100%	100%	100%
吞吐量 (req/s)	12.13	29.21	40.22	19.84	39.97
缓存命中率	82.0%	83.0%	81.8%	95.7%	85.0%
实际 Prefill token 数	583,552	549,376	589,568	137,723	484,608
TTFT p50 (s)	67.15	17.40	7.61	3.53	6.10
TTFT p95 (s)	132.46	37.02	18.91	70.65	22.27

端到端 p95 (s)	133.66	38.93	21.87	73.08	24.91
TPOT p95 (s)	0.0134	0.0178	0.0271	0.0207	0.0264
dispatch_lag p95 (s)	0.0021	0.0025	0.0020	0.0018	0.0022
client_queue p95 (s)	0.0004	0.0004	0.0004	0.0004	0.0004
后端 0/1/2/3 请求数	518/517/504/509	525/512/512/499	507/511/520/510	384/282/194/ 1188	529/525/499/495
最大后端占比	25.3%	25.6%	25.4%	58.0%	25.8%
GPU 利用率 (均值/最大)	94.7% / 100%	87.0% / 100%	81.5% / 100%	44.6% / 100%	84.9% / 100%

(GPU 利用率由实验期间每 2 s 采样一次 nvidia-smi 得到；每轮的汇总写在 run_meta.json

的 gpu_summary 中，随包的原始采样文件 report/run/gpu_samples.jsonl 只对应最后一次运行。)

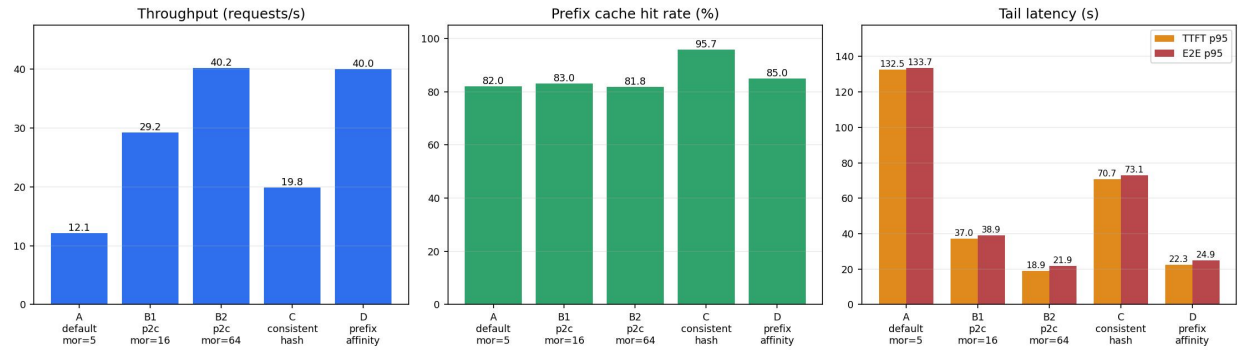


图 2 吞吐量、前缀缓存命中率与尾延迟对比

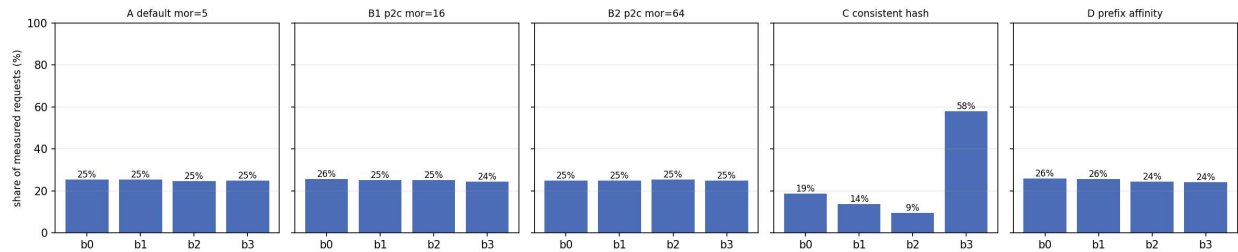


图 3 四个 SGLang 后端的请求占比：C 组出现明显热点（58% 落在一个后端）

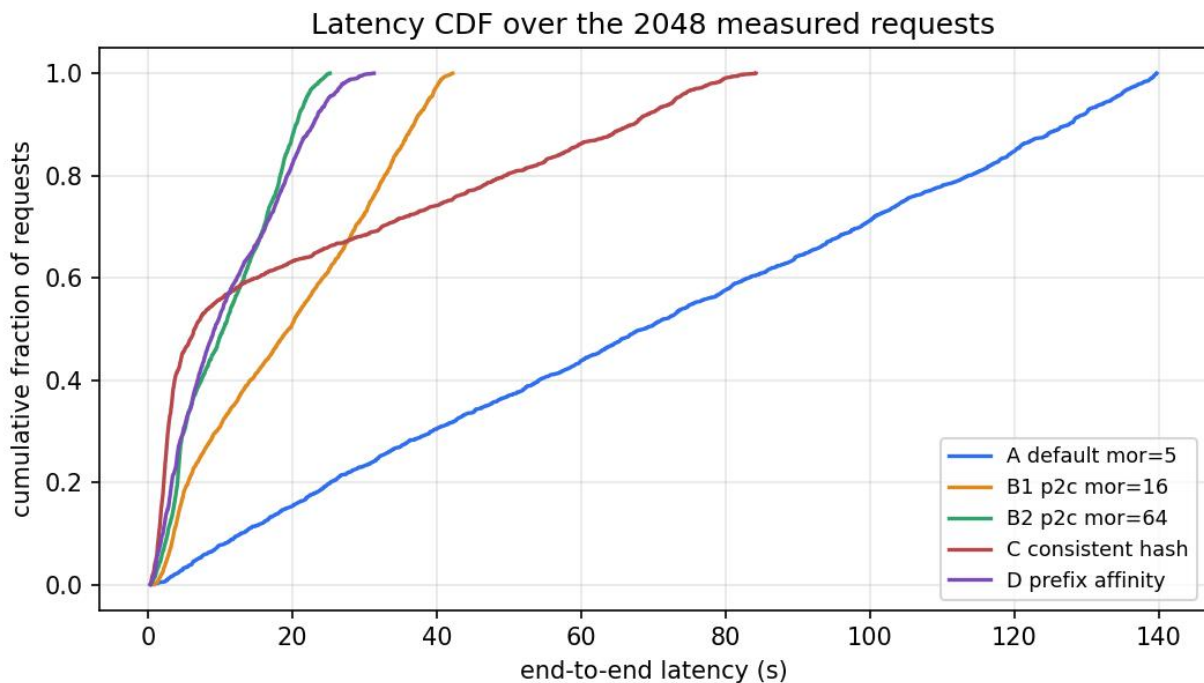


图 4 2048 条测量请求的端到端延迟 CDF: A 组整体右移, C 组呈双峰, B2/D 最集中

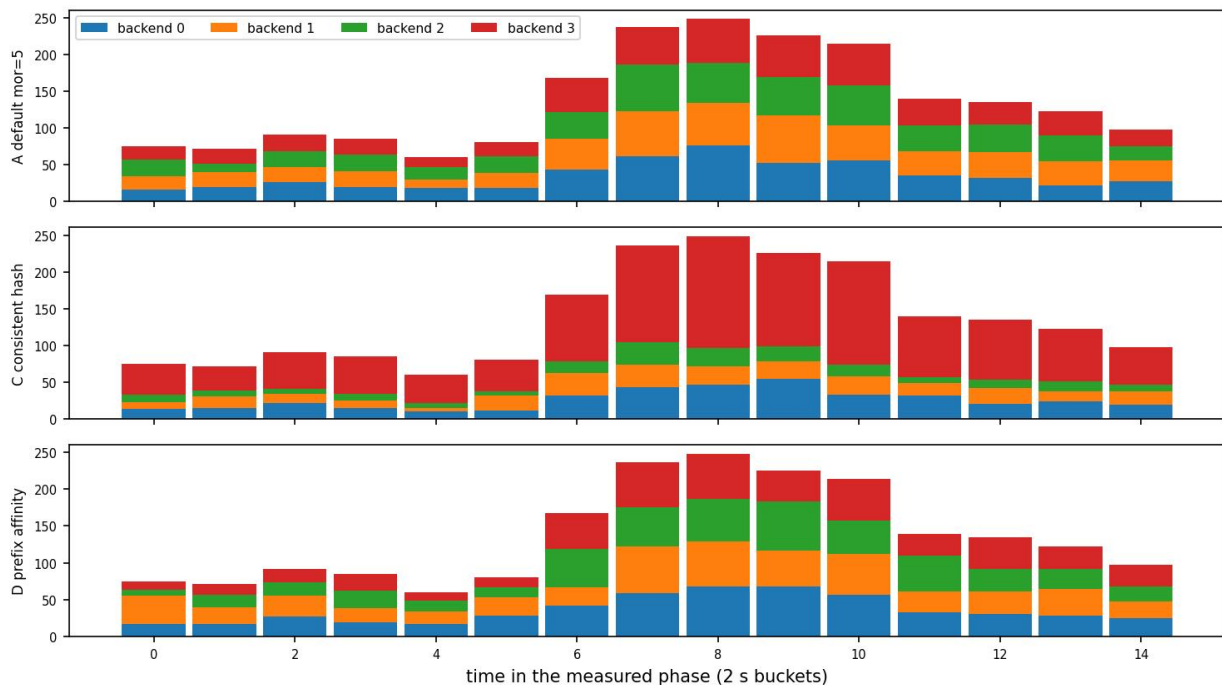


图 5 后端请求随时间 (2 s 桶) 的分布: C 组的热点后端在每个阶段都被持续压满

2.1 分阶段 (steady / burst / recovery)

组别	阶段	请求数	缓存命中率	TTFT p95 (s)	端到端 p95 (s)
A	steady	512	71.3%	35.5	37.7

A	burst	1024	85.9%	110.4	111.7
A	recovery	512	84.7%	137.2	138.8
B1	steady	512	76.0%	6.3	8.7
B1	burst	1024	86.0%	28.4	30.8
B1	recovery	512	84.1%	38.6	40.6
B2	steady	512	72.3%	0.5	4.8
B2	burst	1024	85.2%	16.2	19.6
B2	recovery	512	84.4%	20.8	23.6
C	steady	512	95.5%	39.1	42.1
C	burst	1024	95.8%	72.5	74.9
C	recovery	512	95.9%	71.6	74.3
D	steady	512	92.3%	1.4	4.9
D	burst	1024	82.7%	22.9	26.1
D	recovery	512	82.3%	23.3	25.9

3. 分析

3.1 默认值与 B 选定值为什么表现不同

A 与 B 的唯一差别是 `max_ongoing_requests` ($5 \rightarrow 16 \rightarrow 64$)。这个参数限制的是**每个 Replica

上同时在处理的请求数**，因而也限制了整个服务的请求级并发与调度容量： $4 \text{ 个 Replica} \times 5 = 20$ 。

- 负载的发送窗口只有 29.5 s，而实测单请求平均占用 Replica 槽位约 1.66 s ($2048 \times 1.66 \text{ s} \approx 3390$ 「槽位秒」，除以 20 个槽位 $\approx 170 \text{ s}$)，与 A 组实测的 168.9 s 总耗时几乎一致。也就是说，A 组的总耗时几乎完全由「20 个槽位 $\times$ 排队深度」这一调度容量约束决定：请求在 Ray Serve 侧排队等待槽位，端到端时间里绝大部分是等待，而不是服务时间——A 组 TTFT p50 67.2 s、p95 132.5 s 正是这一等待的直接后果。
- 提高并发上限后，请求更快被分派到四个后端、单副本批次更小，排队迅速变短：TTFT p50 从 67.15 s 降到 7.61 s，p95 从 132.46 s 降到 18.91 s，吞吐提升 3.3 倍。
- 批处理形态确实随之改变，但**这只能作为旁证，不能单独用来断定瓶颈位置**：A 组 GPU 平均利用率 94.7% 而吞吐只有 12.1 req/s，B2 组利用率 81.5% 而吞吐 40.2 req/s；同时 A 组 TPOT p95 (13.4 ms) 低于 B2 (27.1 ms)。这组现象说明「高利用率 $\neq$ 高吞吐」，以及两组的批次构成不同 (A 组槽位少，单次分配给后端的请求更多、批次更大更长)，但利用率与 TPOT 都不足以排除 GPU 计算是限制因素，因此本文只把「并发与调度容量」作为已证实的直接原因。

- 继续抬到 128 (B3)：吞吐 39.92 req/s、TTFT p95 18.93 s、端到端 p95 21.28 s，与 64 基本一致，

说明 64 已在吞吐平台期；再增加并发只会提高内存与尾延迟风险而不带来收益，因此 B 组选定 64。

3.2 前缀亲和为什么同时带来缓存收益和热点

负载的 64 个前缀族各有一段 ≥ 1024 token 的共享前缀，而 SGLang 的 radix cache 只在**同一个后端内部**复用，因此路由策略直接决定缓存能否命中。

- **缓存收益**：C 组把同一前缀族的请求严格绑定到一个 Replica (`num_fallback_replicas=0`)，该族的共享前缀只会在一个后端里被计算一次。实测缓存命中率 95.7%、实际 Prefill token 仅 137,723 (A 组的 23.6%)，稳态阶段 TTFT p50 低至 3.53 s，比 B2 快一倍以上。
- **热点代价**：前缀族的请求量极不均匀——超热族占 768 条 (37.5%)。一致性哈希把它整体映射到后端 3，于是后端 3 收到 1188 条 (58.0%)，后端 2 只有 194 条 (9.5%)。热点后端排队不断累积，C 组因此呈现典型的「双峰」延迟：命中缓存的请求极快 (p50 3.53 s)，排在热点后端后面的请求极慢 (p95 70.65 s、最大 74+ s)，吞吐被最慢的后端拖到 19.8 req/s，GPU 平均利用率只有 44.6%。
- 换言之，前缀亲和把「缓存收益」与「负载偏斜」绑在同一个机制里：**绑定越严格，缓存命中越高，被绑定的那台机器越容易成为瓶颈**。C 组是这个权衡的极端点（严格亲和、零回退）。

3.3 D 使用了哪些信号与规则，改善了什么，付出了什么代价

D 组的路由器 `PrefixAffinityRouter` (`src/target3/hw3/routers.py`) 只使用「当前请求 + 当前服务状态」，不看任何未来请求：

信号

5. 当前请求的前缀族 ID (来自 `X-Session-Id` / `X-Prefix-Family`，Ray Serve 以 `session_id` 暴露给路由器)；
6. 每个 Replica 的队列长度 (Ray Serve 路由器本地缓存的 `queue_len`)；
7. 路由器自己维护的每副本在途请求计数 (在 `on_request_routed` / `on_request_completed` 生命周期钩子中增减)；
8. 每个 Replica 的 `max_ongoing_requests` 背压上限。

规则

9. **亲和**：用 rendezvous hashing (`sha256("hw3:" + 前缀族 + 副本 ID)` 取分数最高者) 为每个前缀族算出稳定的 home 副本。哈希只依赖族 ID 与当前副本集合，副本集合不变则 home 不变，因此同一族的请求会反复命中同一后端的 radix cache；

10. **负载保护**：若 home 副本的压力（在途计数与实测队列长度的较大者）达到 $0.3 \times \text{max_ongoing_requests}$ ，就把请求交给压力最小的其他副本；
11. **绝不丢弃**：所有副本都满时返回「home 优先、其余按压力排序」的分层候选列表，交给 Ray Serve 避免重试。

改善（与 B2 对比，两者 `max\_ongoing\_requests` 相同）

- 缓存命中率 81.8% → 85.0%，实际 Prefill token 589,568 → 484,608（**减少 17.8%**）；收益主要来自那些本来会被随机打散到四个后端、前缀要重复计算四次的族；
- 吞吐 40.22 → 39.97 req/s（差异在单轮波动范围内），没有牺牲吞吐；
- TTFT p50 7.61 → 6.10 s；
- 四个后端请求数 529/525/499/495，最大占比 25.8%，没有出现 C 组那样的热点。

代价

- TTFT p95 18.91 → 22.27 s、端到端 p95 21.87 → 24.91 s（约 **+3.4 s / +3.0 s**）：为了保住亲和，home 副本在阈值附近会多排一点队，这部分延迟就是缓存收益的价格；
- TPOT p95 基本不变（0.0271 → 0.0264 s），说明代价来自排队而非单步解码变慢；
- 路由器需要额外状态（每副本在途计数、请求归属表），实现与调试复杂度高于内置路由器。

D 的参数敏感性（探索性观察，不作为结论）：把保护阈值从 0.3 放松到 0.7

(results/D_variants/relaxed-0.7/)，缓存命中率从 85.0% 降到 83.6%、Prefill token 从 484,608 升到 531,839，而吞吐与尾延迟几乎不变（39.79 req/s、TTFT p95 22.72 s、端到端 p95 25.33 s）。**只能把这看作一次探索性观察**：两个阈值各只跑了一轮，而且本路由器用 Replica ID 参与 rendezvous 哈希，每次重新部署都会改变 Replica ID，从而改变热点前缀族落到哪个副本上，单轮结果无法分离「阈值效应」与「映射变化」。要得到稳定规律，需要对同一配置多轮重复实验（见 README 中 run-1/、run-2/ 的做法），本文不据此下结论。

3.4 客户端与调度端是否成为瓶颈

五组的 dispatch_lag_s p95 均 ≤ 2.5 ms、client_queue_s p95 均为 0.4 ms，说明 2048 条请求基本按计划到达时间发出，压测端没有成为主要瓶颈，表中的差异来自服务端路由而不是客户端。各组均为 --

policy serve --max-in-flight 2048 --timeout 300, 实测最大端到端延迟 141 s (A 组), 未触发超时; 五组 failed=0、incomplete=0。

3.5 数据一致性与可追溯

- 五组共用同一负载 (SHA-256 校验)、同一模型、同一 SGLang 参数、同一四副本拓扑, 只有 Ray Serve 路由配置不同;
- **元数据勘误**: config.json / summary.json 顶层的 model 字段由流量发生器写出, 它没有 --model 参数, 因此记录的是内置默认值而非真实模型; 正式组实际服务

Qwen/Qwen3-4B-Instruct-2507, 正式组后端的 --max-running-requests 为 32。

这两点依据启动日志更正, 更正方式与原文都记录在 results/target3/NOTES.md;

- 每轮开始前: 停止上一轮 Serve 应用 → 用本组配置重新 serve.run → 校验 [4 副本 / 4 节点 / 4 后端] → 对四个后端逐个 POST /flush_cache 并确认 200 → 再开始预热与测量;
- 报告中的每个数字都由 src/target3/hw3/make_numbers.py、aggregate.py 直接从 results/ 下的 requests.csv / summary.json 计算, 可用课程自带 compare_runs.py (输出见 results/target3/comparison/comparison.json) 复核;
- 轮次说明: A/B1/B2/B3/C/D/D' 各 1 轮, 属单轮实验, 报告未对轮间波动做统计推断;
- 判断 max_running_requests 的影响时, 本文比较过 32 与 256 两种后端配置下的 A 组结果

(12.13 与 12.08 req/s), 说明该参数在本负载下不是主导因素; **正式五组统一使用 32**,

依据是四个后端的启动日志 (见 1.5 与 results/target3/NOTES.md)。

4. 复现方式

下面命令按交付包的目录结构书写 (包内根目录即 HW3-182500116 黄传鑫/):

```
# 0) 依赖: torch 2.9.1+cu128 / sglang 0.5.6.post2 / ray[serve] 2.59.0 / aiohttp
# 见 src/target3/requirements.txt
# 1) 起逻辑集群 (1 head + 4 worker) 与 4 个 SGLang 后端, 等待全部 /health 通过
bash src/target3/scripts/start_experiment.sh
python -m hw3.smoke_test          # 需 PYTHONPATH=src/target3; 自检节点数/健康/清缓存/SSE/前缀缓存命中
# 2) 逐组实验: 部署 → 校验映射 → 清四个后端缓存 → 回放 2048 条 → 校验结果
cd src/target3
bash scripts/run_group.sh A_default    A_default_run1    ../../results/target3/A_default
```

```
bash scripts/run_group.sh B1_candidate
B1_max16_run1    ../../results/target3/B_candidates/candidate-1
bash scripts/run_group.sh B2_candidate
B2_max64_run1    ../../results/target3/B_candidates/candidate-2
bash scripts/run_group.sh C_affinity    C_affinity_run1 ../../results/target3/C_affinity
bash scripts/run_group.sh D_improved    D_improved_run1 ../../results/target3/D_improved
```

官方负载脚本（src/target3/workload/，从课程仓库原样复制）与固定负载数据文件的获取、

校验命令见顶层 README.md。

5. 附录

- 逐请求原始数据：results/target3/*/requests.csv（2048 行，含 ttft_s、tpot_s、latency_s、cached_tokens、replica_id、ray_node_id、backend 等字段）
- 每轮命令与元数据：results/target3/*/config.json（含完整 sglang_command）；部署校验与 replica→node→backend 映射：results/target3/*/deploy.json；GPU 采样汇总与验收结论：results/target3/*/run_meta.json
- 记录勘误与核对方法：results/target3/NOTES.md
- 汇总表：results/target3/comparison/table_main.{csv,md}、table_phases.*、table_tiers.*、table_backends.*、summary_all.json、deltas_vs_A.json、comparison.json
- 集群拓扑与自检记录：results/target3/evidence/topology.json、results/target3/evidence/smoke.json
- GPU 采样：逐轮汇总在各自 run_meta.json 的 gpu_summary；随包的原始采样

results/target3/evidence/gpu_samples.jsonl 对应最后一次运行

6. 作业感受

姓名：黄传鑫 学号：182500116

1. 在本次挑战中，AI起到了什么作用？

本次挑战中，AI主要承担了辅助编码、资料检索、故障排查和数据分析的作用。实验开始时，我利用 Codex 梳理 Ray Serve 自定义请求路由器的接口，并将作业要求拆分为集群搭建、后端部署、路由配置、负载回放和结果汇总等步骤。在实验过程中，AI协助编写和检查自动化脚本，分析模型下载、依赖冲突、磁盘空间以及实验元数据不一致等问题。完成实验后，AI还帮助我从逐请求数据中整理吞吐量、缓存命中率、TTFT和后端负载分布。不过，AI给出的建议并不总是正确，因此我通过启动日志、原始实验数据和对照实验逐项核实。对我而言，AI更适合作为提高效率和提供排查思路的辅助工具，最终的实验设计、结果判断和结论仍需要由使用者负责。

2. 本次作业中最困难的部分是什么？你如何克服？

本次实验没有遇到特别大的困难。出现问题时，我用清华镜像处理，并重测了数据。

3. 从0到1开展科研工作需要哪些步骤？你还想进一步研究什么？

首先要调研一下想做的方向目前面临哪些比较大的问题、已有的解决方案，看看能不能再提出新的方案进行优化，或者把别人的 idea 用到新的场景中，也就是“新瓶装旧酒”。其次要看最近的论文，这样才有故事能讲。